

Herman - Project Report

DAT076 / DIT127 Web Applications

norenhe@chalmers.se

June 11, 2026

1 Introduction

The project is a key-ordering system for a locksmith business. A locksmith managing master-key systems (“lock systems”) for buildings uses it to track which customers have access to which lock systems, and customers use it to order copies of keys they are entitled to.

There are two roles:

- **Admins** (locksmith staff) create lock systems and keys, create user accounts, assign lock systems to users, and process incoming key orders by advancing their status.
- **Users** (customers) see only the lock systems assigned to them, place key orders with a quantity and a reason (e.g. lost, damaged), and follow the status of their own orders.

Git repository: <https://github.com/HermanNoren/Herman-DAT076-Project>

Demo: <https://www.youtube.com/watch?v=qTssqkHIZp8>

As I did this as an individual assignment, I have not done an individual evaluation nor written a section in the report about team member responsibilities.

2 Use cases

#	Use case	Role
1	Log in with email and password	both
2	Log out	both
3	View lock systems (admin: all, user: assigned only)	both
4	View a lock system’s details and its keys	both
5	Create a lock system	admin
6	Create a key within a lock system	admin
7	Create a user account (set name, email, password, role)	admin
8	Delete a non-admin user	admin
9	Assign a lock system to a user	admin
10	Unassign a lock system from a user	admin
11	View all users and their assignments	admin
12	Place a key order (quantity + reason)	user
13	View orders (admin: all, user: own history)	both
14	Advance an order’s status (placed → ready → collected)	admin

3 User manual

3.1 Installation

Prerequisites: Node.js ≥ 20 , Docker.

- Start the database (first time only):
`docker run -env POSTGRES_HOST_AUTH_METHOD=trust -publish 5433:5432 -name web_apps_db -detach postgres:17`
- On later runs: `docker start web_apps_db`
- Backend: `cd server`, then `npm install`
- Create `server/.env` containing `SESSION_SECRET=<any secret string>` and `DATABASE_URL="postgres://postgres@localhost:5433"`
- Create the tables (first time only): `npx drizzle-kit migrate`
- Start the backend: `npm run dev`. Runs on `http://localhost:8080` and seeds the demo data
- Frontend (in a separate terminal): `cd client`, then `npm install` and `npm run dev`. The app is accessed at `http://localhost:5173`

The database will be seeded with some demo data, including two accounts:

Account	Email	Password	Role
Alice Admin	alice@example.com	password	admin
Ulf User	ulf@example.com	password	user

3.2 Using the app

3.2.1 Signing in

Open the app and enter your email and password. Customers can not register themselves as this is a system limited to already existing customers, therefore accounts can only be created by an admin.

3.2.2 As a customer (user)

- **Lock Systems** shows the systems you have been given access to. Open one to see its keys.
- Press **Order** next to a key, choose a quantity and a reason (Lost, Damaged, Additional copy, Stolen, or Other with a free-text explanation), to place the order.
- **Orders** shows your order history and each order's current status: *Placed*, *Ready for pickup* or *Collected*.
- Log out from the menu in the bottom-left corner.

3.2.3 As an administrator

- **Lock Systems**: create new systems with **Add Lock System**; open a system and use **Add Key** to add keys (label, description, access level).
- **Orders** lists every order from every customer. Change an order's status with the dropdown as you process it.
- **Users**: create accounts with **Add User**, use the ... button on a user row to assign and unassign lock systems, delete a user with the trash button (admin accounts cannot be deleted).

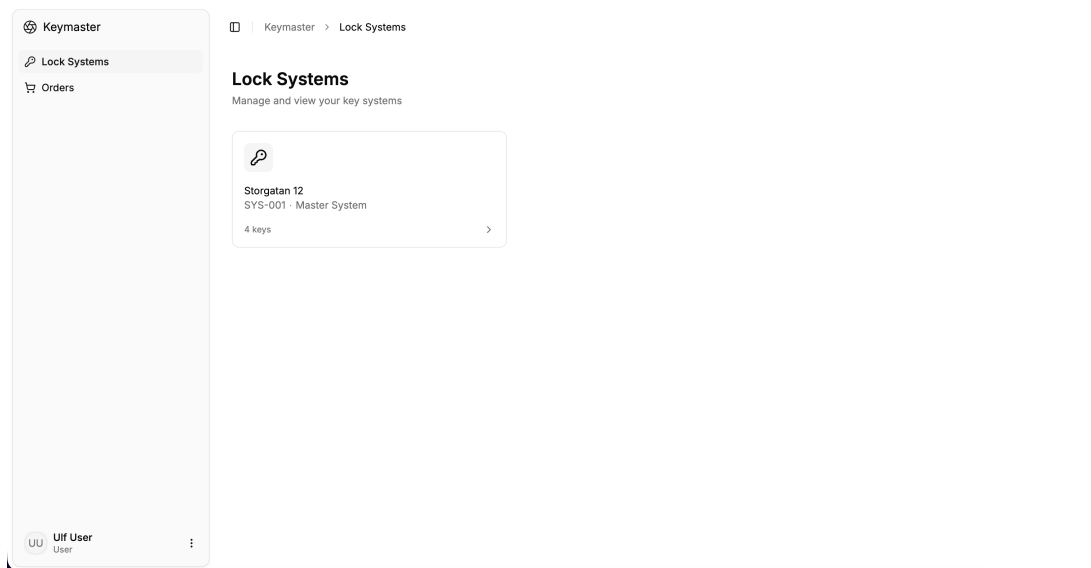


Figure 1: The main page from a customers point of view

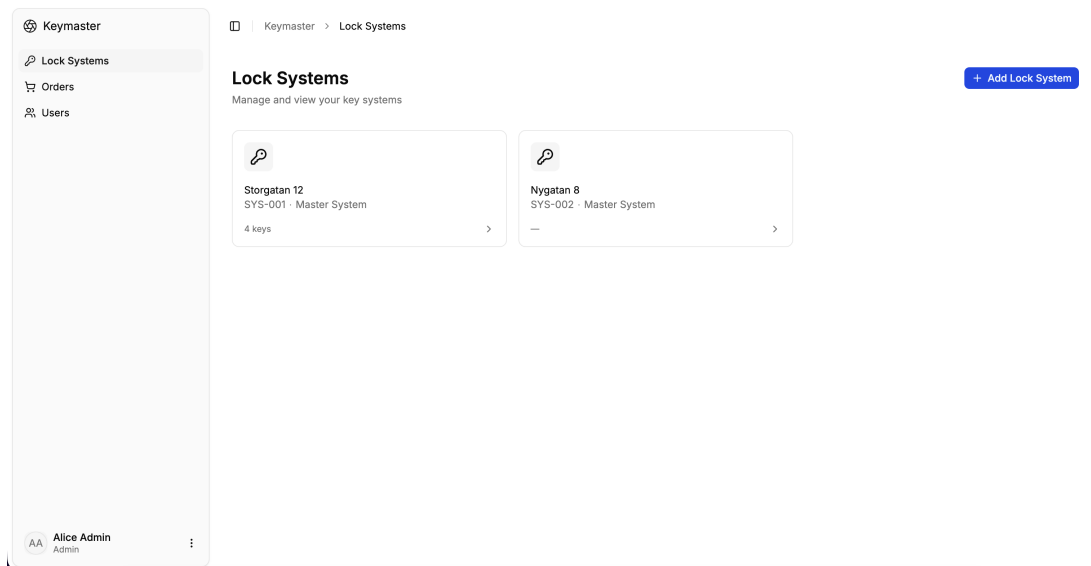


Figure 2: The main page from an admins point of view

4 Design

4.1 Stack

Frontend: React, TypeScript, Vite, Tailwind CSS v4, shadcn/ui, react-hook-form + zod, Axios, React Router, Jest + React Testing Library.

Backend: Node.js, Express, TypeScript, express-session, bcrypt, Drizzle ORM, PostgreSQL (Docker), Jest + Supertest + PGLite (in-process Postgres for tests).

Deviations from the course default stack: Due to previous experience, I have chosen to use some other libraries than the ones taught in the course. Drizzle ORM instead of Sequelize and Tailwind instead of Bootstrap. I've also used shadcn/ui as a UI component system. The lab requirements are met equivalently.

4.2 Architecture overview

- **Sessions:** logging in sets a session cookie. The server derives the user's identity from the session on every request so the client never has to send user IDs.
- **Authorization** is enforced server-side by two guards: `requireAuth` (valid session, else 401) and `requireAdmin` (admin role, else 403). The frontend additionally hides admin-only controls and guards routes (`ProtectedRoute` redirects logged-out visitors to the login page, `AdminRoute` redirects non-admins away from the admin-only Users page) but this is UX only, the API is the real security boundary.
- **Backend:** routers (HTTP, validation, status codes) → services (business logic, defined by interfaces) → Drizzle → PostgreSQL. The concrete service implementations are instantiated in a single composition root (`server/src/service/index.ts`); routers depend only on the service interfaces (see Section 4.6).
- **Frontend:** pages are just thin composition shells, feature hooks own data fetching/mutations, feature components own rendering (see Section 4.3).

4.3 Frontend

The frontend is organised by feature (`src/features/<domain>/{components, hooks}`), with shared layout components in `src/components/` and pages in `src/pages/`. Backend calls below are made through the feature hooks listed alongside each table.

4.3.1 Pages

- **LoginPage:** centers the login card (`LoginForm`). Makes no backend calls itself.
- **LockSystemsPage:** lists the lock systems visible to the logged-in user; admins also get a create button. Fetches `GET /lock-systems` and `GET /keys` (for the key counts).
- **LockSystemDetailPage:** shows one lock system and its keys table. Fetches `GET /lock-systems/:referenceCode` and `GET /keys?lockSystemId=`.
- **OrdersPage:** admins see all orders with status controls, users see their own order history. Fetches `GET /orders`, joined with key/system/user display data by `useOrderRows`.
- **UsersPage:** admin-only user management, wrapped in `AdminRoute`. Fetches `GET /users` and `GET /lock-systems`.

4.3.2 Feature: auth

- **AuthProvider:** context provider wrapping all routes. Holds the session user as state (`user`, `isLoading`), restores the session on mount (`GET /session`) and exposes `login` (`POST /session`) and `logout` (`DELETE /session`). Props: `children`.
- **LoginForm:** email/password form with validation. Takes no props. Holds react-hook-form state. Submits via `useLogin`.
- **ProtectedRoute** and **AdminRoute:** route guards used by the route table in `App.tsx`. The first renders nothing while the session check is loading and redirects logged-out visitors to `/login`; the second additionally redirects non-admin users to the start page and wraps the `/users` route. Props: `children`.

Hooks:

- `useAuth()`: accesses the auth context (current user, login, logout).
- `useLogin()`: logs in (POST `/session`) and redirects into the app on success. Exposes `isLoading` and `error` for the form.

4.3.3 Feature: lock-systems

- `LockSystemList`: grid of lock system cards. Props: `lockSystems`, `keyCounts?`.
- `LockSystemCard`: one lock system, linking to its detail page. Props: `lockSystem`, `keyCount?`.
- `LockSystemKeys`: the keys table of one system, users get an Order button per key. Props: `lockSystem`, `keys`.
- `CreateLockSystemSheet`: admin sheet for creating a lock system. Props: `onCreated`. State: open/closed and react-hook-form state. Submits POST `/lock-systems` via `useCreateLockSystem`.
- `CreateKeySheet`: admin sheet for creating a key. Props: `lockSystemId`, `onCreated`. State: open/closed and react-hook-form state. Submits POST `/keys` via `useCreateKey`.

Hooks:

- `useLockSystems`: GET `/lock-systems` (visible systems to the current user).
- `useLockSystem(ref)`: GET `/lock-systems/:referenceCode`.
- `useLockSystemKeys(id)`: GET `/keys?lockSystemId=`.
- `useKeyCountsByLockSystem`: GET `/keys`, maps key count per system.
- `useCreateLockSystem`, `useCreateKey`: the create mutations used by the sheets above.

4.3.4 Feature: users

- `RegularUserList`: table of customers with assign and delete actions. Props: `users`, `lockSystems`, `onAssigned`, `onDeleted`. Deletes via `useDeleteUser` (DELETE `/users/:id`).
- `AdminList`: read-only table of admin accounts. Props: `admins`.
- `CreateUserSheet`: admin sheet for creating a user account. Props: `onCreated`. State: open/closed and react-hook-form state. Submits POST `/users` via `useCreateUser`.
- `AssignLockSystemSheet`: toggles a user's lock-system assignments. Props: `user`, `lockSystems`, `onAssigned`. State: `assignedIds`. Calls PATCH `/users/:id/assign-lock-system` and PATCH `/users/:id/unassign-lock-system` via `useAssignLockSystem` and `useUnassignLockSystem`.

Hooks:

- `useUsers(enabled?)`: GET `/users`, can be disabled for non-admin viewers.
- `useGroupedUsers`: wraps `useUsers` and splits the result into customers and admins.
- `useCreateUser`, `useAssignLockSystem`, `useUnassignLockSystem`, `useDeleteUser`, the mutations used by the components above.

4.3.5 Feature: orders

- **PlaceOrderSheet**: order form: quantity, reason, and a free-text detail field when the reason is “other”. Props: `keyItem`, `lockSystem`. State: open/closed and react-hook-form state. Submits `POST /orders` via `usePlaceOrder`.
- **AdminOrderList**: all orders with a status dropdown per row. Props: `rows`, `onStatusChanged`. Updates via `useUpdateOrderStatus` (`PATCH /orders/:id/status`).
- **UserOrderList**: the customer’s own orders with status badges. Props: `rows`.

Hooks:

- `useOrders`: `GET /orders` (all for admins, own for users).
- `useAllKeys`: `GET /keys`, used to join orders with key data.
- `useOrderRows({withUsers})`: combines `useOrders`, `useAllKeys`, `useLockSystems` and (for admins) `useUsers` into display rows.
- `usePlaceOrder`, `useUpdateOrderStatus`, the mutations used by the components above.

4.3.6 Shared components

- **AppLayout**, **AppHeader**, **AppSidebar**: page layout, sidebar navigation and header around the routed page content.
- **NavUser**: sidebar footer showing the logged-in user with a log-out menu. Props: `user`.
- **PageHeader**: page title, description, and an optional action slot used for the admin-only create buttons. Props: `title`, `description`, `action?`.
- **ui/***: the shadcn/ui primitives (`Button`, `Input`, `Sheet`, `Select`, `Field`, etc...) used by everything above.

4.4 Backend

All endpoints except `POST /session` require a valid session cookie (**401 Unauthorized** otherwise). Endpoints marked **admin only** additionally require `role === "admin"` (**403 Forbidden** otherwise). Validation errors return **400** with a plain-text message; unexpected errors return **500**.

4.4.1 Session

- `POST /session`: log in. Body: `{email, password}`. Success: 200 with `UserPublic`. Errors: 400 missing fields, 401 invalid credentials (does not reveal whether email or password was wrong).
- `GET /session`: return the currently logged-in user. Success: 200 with `UserPublic`. Errors: 401 when no session exists.
- `DELETE /session`: log out and destroy the session. Success: 204, no body. Errors: 401 when no session exists.

4.4.2 Lock systems

- GET `/lock-systems`: list visible lock systems. All of them for admins, only assigned ones for users. Success: 200 with `LockSystem[]`. Errors: 401.
- GET `/lock-systems/:referenceCode`: one lock system by its reference code. Success: 200 with `LockSystem`. Errors: 401, 404 lock system not found.
- POST `/lock-systems` (admin only): create a lock system; the next `SYS-xxx` reference code is assigned automatically. Body: `{name, description}`. Success: 201 with the created `LockSystem`. Errors: 400 missing/invalid fields, 401, 403.

4.4.3 Keys

- GET `/keys`: list all keys; with an optional `?lockSystemId=` query parameter, only that system's keys. Success: 200 with `Key[]`. Errors: 401, 404 lock system not found (when the parameter is given).
- POST `/keys` (admin only): create a key in a lock system. Body: `{label, description, accessLevel, lockSystemId}`. Success: 201 with the created `Key`. Errors: 400 invalid fields or access level, 401, 403, 404 lock system not found, 409 duplicate label within the same lock system.

4.4.4 Users

- GET `/users` (admin only): list all users. Success: 200 with `UserPublic[]`. Errors: 401, 403.
- POST `/users` (admin only): create a user account. Body: `{name, email, password, role}`. Success: 201 with the created `UserPublic`. Errors: 400 empty name, password shorter than 8 characters, or invalid role, 401, 403, 409 duplicate email.
- DELETE `/users/:id` (admin only): delete a non-admin user. Their assignments and orders are removed by the database cascade. Success: 204, no body. Errors: 401, 403 caller is not an admin *or* the target is an admin, 404 user not found.
- PATCH `/users/:id/assign-lock-system` (admin only): assign a lock system to a user. Body: `{lockSystemId}`. Success: 200 with the updated `UserPublic`. Errors: 400 missing `lockSystemId`, 401, 403, 404 user not found or lock system not found.
- PATCH `/users/:id/unassign-lock-system` (admin only): remove a lock system assignment from a user. Body: `{lockSystemId}`. Success: 200 with the updated `UserPublic`. Errors: 400 missing `lockSystemId`, 401, 403, 404 user not found or lock system out found.

4.4.5 Orders

- GET `/orders`: list orders. All of them for admins, only their own for users. Success: 200 with `Order[]`. Errors: 401.
- POST `/orders`: place an order for the logged in use. The user is taken from the session, never from the body. Body: `{keyId, quantity, reason, reasonDetail?}`. Success: 201 with the created `Order` (status placed). Errors: 400 quantity below 1, invalid reason, or missing `reasonDetail` when the reason is "other" 401, 403 when the key belongs to a lock system the user is not assigned to, 404 key not found or user not found.

- PATCH /orders/:id/status (admin only): update an order's status. Body: {status}. Success: 200 with the updated Order. Errors: 400 invalid status, 401, 403, 404 order not found.

4.5 Database

The database has five tables: `users`, `lock_systems`, `keys`, `user_lock_systems` (join table), and `orders`. Relationships:

- a lock system contains many keys.
- users and lock systems are linked many-to-many through `user_lock_systems`.
- a user places many orders, each order is for exactly one key.

Constraints and other design decisions in the database:

- Enums (`user_role`, `access_level`, `order_reason`, `order_status`), invalid values are rejected.
- Composite unique on `keys` (`lock_system_id`, `label`), key labels are unique per lock system.
- Check quantity > 0 on orders.
- ON DELETE CASCADE: Deleting a user removes their assignments and order history. This might not be the best way to do it as in reality orders are tightly coupled with receipts, which are legal documents and should not be modified. A better way to handle it would be to have some sort of soft delete, so users can get their data deleted for GDPR purposes, but orders (and receipts) remain intact. But I figured just deleting is enough for this assignment.

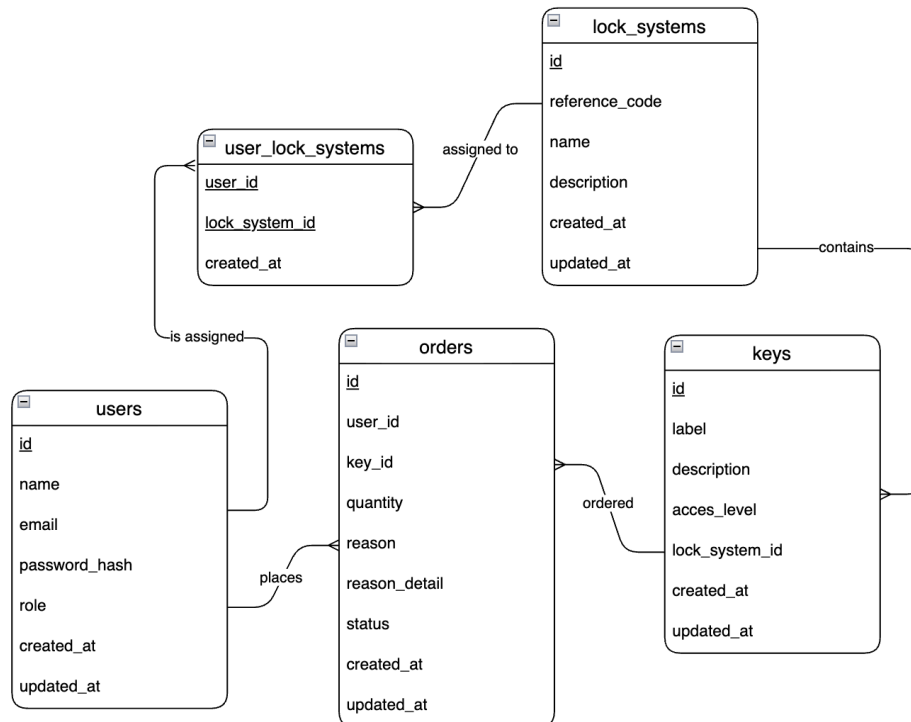


Figure 3: Entity-relationship diagram.

4.6 Design principles

- **Single responsibility:** routers only translate HTTP to service calls, services own the business rules and persistence, and on the frontend pages compose, hooks fetch, components render.
- **Open/closed:** new functionality is added by extension rather than by modifying existing code. A new domain gets its own model, service interface, service implementation and router on the backend, and its own feature folder on the frontend, all following the established patterns. The service interfaces serve the same goal, a new implementation (e.g. a different storage backend) is a new class enabled in the composition root, with no consumer touched. The one place where a change fans out is the shared enums. Adding an order reason touches the model type, the database enum, the router's validation list and the client's labels, a consequence of the client and server not sharing a types package.
- **Liskov substitution:** the tests rely on exactly this. The same service classes run unchanged against an in-process PGLite database instead of the Docker PostgreSQL (Section 4.7).
- **Interface segregation:** the interfaces are small and per-domain rather than one large service interface, and `UserPublic` (the `User` model minus the password hash) is the only user shape that ever leaves the server.
- **Dependency inversion:** every service is defined by an interface and routers depend only on those. The concrete Drizzle-backed implementations are chosen and wired in one composition root, `server/src/service/index.ts`, where `OrderDBService` also gets the user and key services it validates orders against injected through its constructor. Swapping an implementation is a one-line change in the composition root.

4.7 Testing

- **Backend:** Tests per router covering the happy path and error scenarios of every use case, plus a service-level unit test for branches unreachable over HTTP. Tests run the services against **PGLite**, an in-process PostgreSQL, so real constraints/enums/cascades are exercised, each test file gets a fresh migrated and seeded database, and the development database is never touched.
- **Frontend:** Component tests with feature hooks mocked, form validation, submitted values, error display, list rendering, empty states, and the `AdminRoute` guard (an admin sees the page, a regular user is redirected).

4.8 Security considerations

- Passwords are hashed with bcrypt and never leave the server.
- Identity always derived from the server-side session, never client-side user IDs.
- All authorization are made server-side. Hidden actions in the front end is purely UX.
- Login failure does not reveal whether the email or the password was wrong.
- SQL injection prevented by validating inputs in client-side fields as well as Drizzle's parameterized queries.